

目录

事件系统设计（订阅发布模式）2

特效对象池设计（单例模式）2

协程.....4

角色八向移动设计（动画混合树）4

攻击检测设计（动画事件）5

事件系统设计（订阅发布模式）

事件系统主要用于模块间的解耦，这里采用订阅发布模式实现模块间通信的解耦，弊端是当设置监听器但为销毁时会造成内存泄漏且不易发觉。

首先梳理所有的模块，分析哪些模块间存在关联

技能模块 核心机制模块 攻击检测模块 索敌模块 攻击模块

可以得到以下几个事件。

传递“连击数”事件：

订阅：技能模块

发布：核心机制模块

传递参数：float 类型

重置“连击数”事件

订阅：技能模块

发布：核心机制模块

攻击命中事件

订阅：核心机制模块

发布：攻击检测模块

命中检测事件（传递参数用以选择开启碰撞箱）

订阅：攻击检测模块

发布：技能模块 攻击模块

传递参数：float 类型

索敌事件(距离角色最近)

索敌事件(距离屏幕中心最近)

申请索敌事件：

订阅：索敌模块

发布：技能模块

索敌结果返回事件：

订阅：技能模块

发布：索敌模块

传递参数：GameObject 类型

特效对象池设计（单例模式）

由于在 ACT 游戏中需要频繁的进行普通攻击和技能释放，一些特效需要频繁的被生成和销毁造成内存开销，因此这里使用对象池统一管理。以下是简单的对象池实现框架。

```
public class ObjectPool : MonoBehaviour
{
    // 单例实例
    public static ObjectPool Instance { get; private set; }
```

```

// 池配置: 存储预制体
[System.Serializable]
public class Pool
{
    public string tag;           // 对象标识
    public GameObject prefab;    // 预制体
    public int initialSize;      // 初始数量
}

public List<Pool> pools;        // 所有对象池配置
private Dictionary<string, Queue<GameObject>> poolDictionary; // 实际对象池

private void Awake()
{
    // 单例初始化
    if (Instance == null)
        Instance = this;
    else
        Destroy(gameObject);

    // 初始化所有对象池
    InitializePools();
}

// 初始化所有池
private void InitializePools()
{
    poolDictionary = new Dictionary<string, Queue<GameObject>>();

    foreach (Pool pool in pools)
    {
        Queue<GameObject> objectPool = new Queue<GameObject>();

        // 预先生成初始对象
        for (int i = 0; i < pool.initialSize; i++)
        {
            GameObject obj = CreateNewObject(pool.prefab);
            objectPool.Enqueue(obj);
        }

        poolDictionary.Add(pool.tag, objectPool);
    }
}

```

```

// 创建新对象并设置为未激活
private GameObject CreateNewObject(GameObject prefab)
{
    GameObject obj = Instantiate(prefab);
    obj.SetActive(false);
    obj.transform.SetParent(transform); // 可选：将对象设为池的子物体
    return obj;
}
}

```

协程

协程是一种处理异步操作的机制，

启动协程：

通过 `StartCoroutine()` 方法启动。

暂停与恢复：

使用 `yield return` 指定暂停条件，例如：

`yield return new WaitForSeconds(2f);` 暂停 2 秒。

`yield return null;` 等待下一帧。

`yield return new WaitForEndOfFrame;` 在渲染完成后执行。

终止协程：

通过 `StopCoroutine()` 或 `StopAllCoroutines()` 手动停止，或当关联的 `MonoBehaviour` 被销毁时自动停止。

在本项目中处理冲刺逻辑时，冲刺部分代码被封装到一个函数中且脚本结构不允许将冲刺函数用在 `update` 中，因此用到了协程来处理，在不阻塞主线程的条件下，实现了异步处理角色的冲刺逻辑。

角色八向移动设计（动画混合树）

本项目中使用了混合树来实现八向移动。

使用两个 `Float` 类型参数 `Horizontal` 和 `Vertical`（即虚拟轴），在混合树中选择 `2D Simple Directional` 模式来明确方向

为每个动画分配 `Pos X` 和 `Pos Y`，代表输入方向：

动画名称	Pos X	Pos Y
Idle	0	0

动画名称	Pos X	Pos Y
Walk_Forward	0	1
Walk_Back	0	-1
Walk_Left	-1	0
Walk_Right	1	0
Walk_LeftForward	-1	1
Walk_RightForward	1	1
Walk_LeftBack	-1	-1
Walk_RightBack	1	-1

代码中获取虚拟轴为动画参数赋值：

```
// 获取输入
float horizontal = Input.GetAxis("Horizontal");
float vertical = Input.GetAxis("Vertical");

// 归一化输入向量
Vector2 input = new Vector2(horizontal, vertical).normalized;

// 驱动 Animator 参数
animator.SetFloat("Horizontal", input.x);
animator.SetFloat("Vertical", input.y);
```

混合树会根据虚拟轴的输入自动播放对应移动方向的动画。

攻击检测设计（动画事件）

Act 游戏中攻击命中检测是核心功能之一，实现方式有碰撞盒检测和射线检测，射线检测多用于射击游戏，因此采用碰撞盒检测。

碰撞盒检测又分为两种常见的设计：附加到武器上和手动划定攻击区域。附加到武器上的方式简单易实现，但缺点是攻击检测过度依赖武器的挥动。因此采用泛用性更高的手动划定攻击区域。

攻击检测需要精确控制碰撞箱的开关，因此我使用动画事件的方式来实现，将碰撞箱的开关

附加到动画上可以精确控制攻击的开始和结束位置，达到更好的视觉效果。